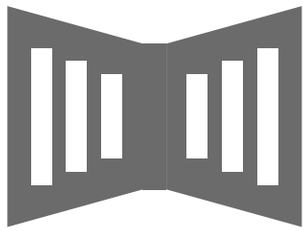
(a) Prediction-first (common practice): build a predictor, then see the gain



request trace



Train a predictor

who returns · when · where to place

`predictor.fit(trace)`

for each returning request

`serve loop, predictor in the loop`

place
evict
prefetch

Predictor-driven policy

`policy(pred)`

load



Restore parked KV

load from CPU / storage

`restore(kv)`



TTFT

report gain

vs. LRU or a weak base

the gain is known only after the predictor has been built

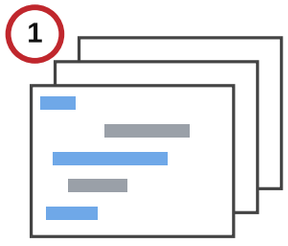
(b) Ours: measure first, bound the value, then decide whether to build

Registered before the run — `registration file`

- thresholds <5% · ≥15%
- rival set per action
- deviation log

thresholds

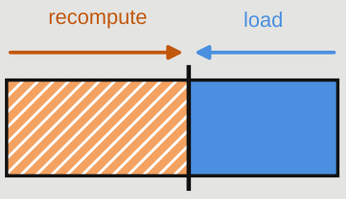
1



request trace

`3 real traces`

for each cell: trace × slow tier × GPU idle/busy × CPU size



Base restore

INT4 + Cake

`same trace, same base, change ONE thing`

FreeInference (agent trace)



gap 0.30%

Mooncake (chat trace)



gap 50.5%

5 **Verdict (registered)**

- < 5% stop
- 5–15% try a free signal
- ≥ 15% worth building

6 **decision per trace**

FreeInference: ≤ 0.30% → skip

Mooncake: ≤ 50.5% → typing hint first

the ceiling is known before any predictor is built

TTFT of returning requests, rival = 100%; gap = 1 − T(oracle) / T(rival); CPU 128 GiB (Mooncake: SSD class, GPU idle)

		common practice (from abstracts, not verified in full text)	ours
1	**Input**	a request trace; the predictor is trained on it	the same trace, with the best fixed restore (INT4 + Cake) as the base
2	**Compare**	against LRU or a base without Cake	against the best non-predictive rival for the same action (LRU, ARC, S3-FIFO ...)
3	**Foresight**	a learned predictor: who returns, when, where to place	an oracle that knows ONE future quantity at a time (prefetch, eviction order, drop)
4	**Free signal**	rare (e.g. tool-reported progress used as a hint)	user starts typing → prefetch; no model needed (0.81 of perfect prefetch)
5	**Decide**	the gain is measured after the predictor exists	ceiling = 1 − T(oracle)/T(rival); thresholds registered before the run
6	**Output**	a predictor and the gain it reports	a decision per trace: FreeInference ≤0.30% → skip; Mooncake ≤50.5% → typing hint